

Question Answering with

Transformers & BERT

A Complete Study Note

Fine-tuning BERT on the SQuAD Dataset
with Trainer API & Custom Accelerate Loop

Includes: Dataset Prep ■ Tokenization ■ Preprocessing ■ Training ■ Evaluation

1. Introduction to Extractive QA

Extractive Question Answering (EQA) is a Natural Language Processing task where a model reads a passage (context) and identifies the exact span of text that answers a given question. Unlike generative QA, no new text is generated — the answer is a substring of the input context.

Task Types — QA Landscape

- Extractive QA (our focus): Answer = span of text within a given context
- Abstractive / Generative QA: Answer is generated freely (T5, BART, GPT)
- Open-Domain QA: Retrieves documents first, then extracts answers
- Multiple-Choice QA: Selects from given options

Encoder-only models like BERT are well-suited for extractive QA because they produce rich contextual representations for every token. The model predicts two positions per token: a start logit and an end logit. The span between the argmax of each gives the predicted answer.

Why BERT for EQA?

- Bidirectional attention: sees both left and right context simultaneously
- Pre-trained on large corpora — rich semantic representations
- Fine-tuning adds a lightweight span-prediction head on top
- Fast tokenizer with offset mappings enables character-to-token alignment

Model Architecture — Span Prediction Head

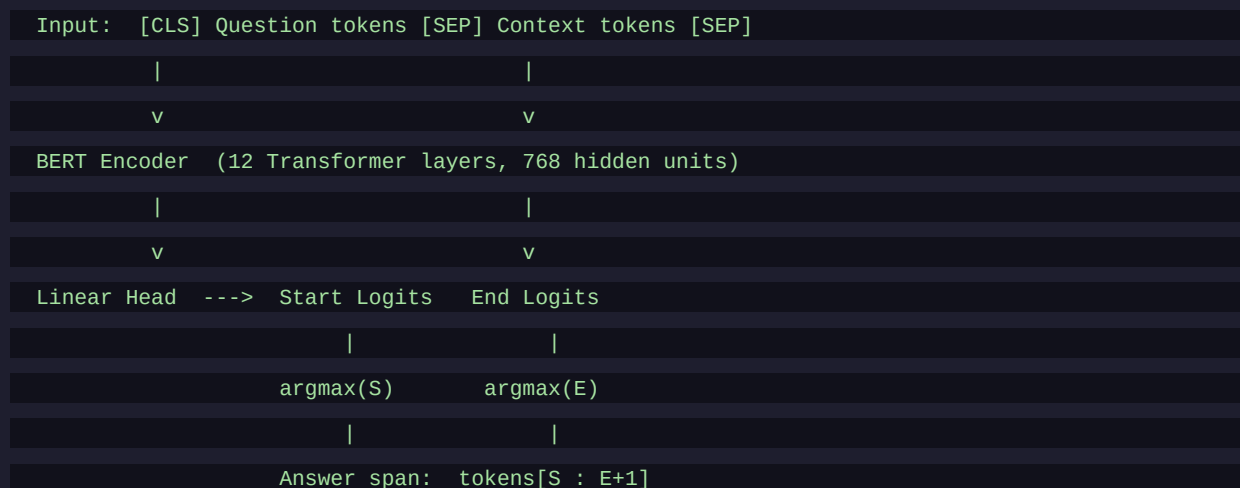


Fig 1. BERT fine-tuned for Extractive QA — outputs start & end logit per token

2. The SQuAD Dataset

The Stanford Question Answering Dataset (SQuAD) is the most widely used academic benchmark for extractive QA. It consists of reading-comprehension questions posed by crowd workers on Wikipedia articles, where the answer to each question is a segment of text from the corresponding passage.

Split	Samples	Answers per Q	Notes
Train	87,599	1 (exact)	Used for fine-tuning
Validation	10,570	1–3 (multiple)	Used for evaluation

Table 1. SQuAD v1.1 dataset statistics

2.1 Loading the Dataset

```
from datasets import load_dataset
```

```
raw_datasets = load_dataset('squad')
```

```
print(raw_datasets)
```

Output:

```
DatasetDict({
  train: Dataset({
    features: ['id', 'title', 'context', 'question', 'answers'],
    num_rows: 87599
  })
  validation: Dataset({
    features: ['id', 'title', 'context', 'question', 'answers'],
    num_rows: 10570
  })
})
```

2.2 Dataset Sample Inspection

```
print('Context: ', raw_datasets['train'][0]['context'])
```

```
print('Question:', raw_datasets['train'][0]['question'])
```

```
print('Answer: ', raw_datasets['train'][0]['answers'])
```

Output:

```
Context: 'Architecturally, the school has a Catholic character...'
```

```
Question: 'To whom did the Virgin Mary allegedly appear in 1858?'
```

```
Answer: {'text': ['Saint Bernadette Soubirous'], 'answer_start': [515]}
```

Key Fields in SQuAD

- context — The passage from Wikipedia
- question — The crowd-sourced question about the context
- answers — Dict with 'text' (answer string) and 'answer_start' (char index)

- id — Unique example identifier (used for evaluation mapping)

During training, each sample has exactly one answer. During validation, multiple answers may exist for each question (up to 3), and evaluation picks the best-matching prediction.

3. Tokenization & Input Format

3.1 Setting Up the Tokenizer

```
from transformers import AutoTokenizer

model_checkpoint = 'bert-base-cased'

tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

print(tokenizer.is_fast)  # Must be True for offset mapping
```

Output:

```
True
```

3.2 Input Structure — [CLS] Question [SEP] Context [SEP]

BERT receives both question and context concatenated with special tokens. The tokenizer handles this automatically when both strings are passed together.

```
inputs = tokenizer(question, context)

tokenizer.decode(inputs['input_ids'])
```

Output:

```
'[CLS] To whom did the Virgin Mary allegedly appear in 1858? [SEP]
Architecturally, the school has a Catholic character... [SEP]'
```

3.3 Handling Long Contexts — Sliding Window

Many contexts exceed the maximum token length (384). A sliding window approach splits long contexts into overlapping chunks so no information is lost. Each chunk becomes a separate training feature.

Parameter	Value	Description
max_length	384	Max tokens per feature
stride	128	Overlap between consecutive chunks
truncation	'only_second'	Only truncate context, never question
return_overflowing_tokens	True	Enable multi-chunk splitting
return_offsets_mapping	True	Map tokens back to character positions

Table 2. Tokenizer parameters for sliding-window chunking

```
inputs = tokenizer(
    question, context,
    max_length=384, truncation='only_second',
    stride=128, return_overflowing_tokens=True,
    return_offsets_mapping=True,
)

# Keys returned:
```

```
# input_ids, token_type_ids, attention_mask,  
# offset_mapping, overflow_to_sample_mapping
```

Output:

```
# 4 examples gave 19 features  
# overflow_to_sample_mapping: [0,0,0,0,1,1,1,1,2,2,2,2,3,3,3,3,3,3]
```

The `overflow_to_sample_mapping` tells us which original example each feature came from. This is critical for evaluation, where we need to merge predictions across all features belonging to the same example.

4. Preprocessing Training Data

Training labels are token indices (start_position, end_position) pointing to where the answer begins and ends within the tokenized input. Finding these requires mapping character-level answer positions to token-level positions using offset mappings.

4.1 Label Strategy

Situation	Label
Answer fully within the context chunk	(start_token_idx, end_token_idx)
Answer not in this chunk (outside range)	(0, 0) → predict [CLS]
Answer partially in chunk (truncated)	(0, 0) → predict [CLS]

Table 3. Label assignment strategy for training features

4.2 Full Preprocessing Function (Training)

```
max_length = 384
stride = 128

def preprocess_training_examples(examples):
    questions = [q.strip() for q in examples['question']]
    inputs = tokenizer(
        questions, examples['context'],
        max_length=max_length, truncation='only_second',
        stride=stride, return_overflowing_tokens=True,
        return_offsets_mapping=True, padding='max_length'
    )
    offset_mapping = inputs.pop('offset_mapping')
    sample_map = inputs.pop('overflow_to_sample_mapping')
    answers = examples['answers']
    start_positions, end_positions = [], []

    for i, offset in enumerate(offset_mapping):
        sample_idx = sample_map[i]
        answer = answers[sample_idx]
        start_char = answer['answer_start'][0]
        end_char = start_char + len(answer['text'][0])
        sequence_ids = inputs.sequence_ids(i)

        # Find context start/end token indices
        idx = 0
        while sequence_ids[idx] != 1: idx += 1
        context_start = idx
        while sequence_ids[idx] == 1: idx += 1
        context_end = idx - 1
```


5. Preprocessing Validation Data

Validation preprocessing is simpler — we don't generate labels. Instead, we store offset mappings (for context tokens only; question offsets are set to None) and example IDs so that, at evaluation time, we can map feature-level predictions back to the original examples and extract answer text.

```
def preprocess_validation_examples(examples):
    questions = [q.strip() for q in examples['question']]
    inputs = tokenizer(
        questions, examples['context'],
        max_length=max_length, truncation='only_second',
        stride=stride, return_overflowing_tokens=True,
        return_offsets_mapping=True, padding='max_length'
    )
    sample_map = inputs.pop('overflow_to_sample_mapping')
    example_ids = []

    for i in range(len(inputs['input_ids'])):
        sample_idx = sample_map[i]
        example_ids.append(examples['id'][sample_idx])

    # Set question offsets to None (only keep context offsets)
    sequence_ids = inputs.sequence_ids(i)
    offset = inputs['offset_mapping'][i]
    inputs['offset_mapping'][i] = [
        o if sequence_ids[k] == 1 else None
        for k, o in enumerate(offset)
    ]

    inputs['example_id'] = example_ids
    return inputs
```

Output:

```
# 10570 examples -> 10822 features (slight increase due to sliding window)
```

Why None for question offsets?

- At post-processing time, we only have the tokenized features — not the raw inputs.
- Setting question-token offsets to None lets us safely skip them when extracting answer spans.
- Only context offsets (sequence_id == 1) are kept; these map token positions to char positions.

6. Post-Processing & Evaluation

The model outputs a start logit and end logit for every token. Post-processing converts these raw scores into a human-readable answer string, then the SQuAD metric computes Exact Match (EM) and F1 score.

6.1 Post-Processing Steps

#	Step	Detail
1	Mask non-context logits	Skip positions where offset is None (question tokens)
2	Select top-N start/end indices	n_best=20 highest logit positions each
3	Filter invalid pairs	end >= start, length <= max_answer_length=30
4	Score each candidate pair	score = start_logit + end_logit (log-sum trick)
5	Pick best answer per example	Aggregate across all features; take argmax score
6	Extract answer text	context[offset[start][0] : offset[end][1]]

Table 4. Post-processing pipeline steps

6.2 compute_metrics() Function

```
import collections, numpy as np
import evaluate

metric = evaluate.load('squad')
n_best = 20
max_answer_length = 30

def compute_metrics(start_logits, end_logits, features, examples):
    # Map example id -> list of feature indices
    example_to_features = collections.defaultdict(list)
    for idx, feature in enumerate(features):
        example_to_features[feature['example_id']].append(idx)

    predicted_answers = []
    for example in examples:
        example_id = example['id']
        context = example['context']
        answers = []

        for feat_idx in example_to_features[example_id]:
            s_logit = start_logits[feat_idx]
            e_logit = end_logits[feat_idx]
            offsets = features[feat_idx]['offset_mapping']
            s_idx = np.argsort(s_logit)[-1:-n_best-1:-1].tolist()
            e_idx = np.argsort(e_logit)[-1:-n_best-1:-1].tolist()

            for si in s_idx:
```


7. Fine-Tuning with Trainer API

The HuggingFace Trainer API abstracts the training loop. We define hyperparameters through TrainingArguments and hand everything off to Trainer.

7.1 Model Instantiation

```
from transformers import AutoModelForQuestionAnswering

model = AutoModelForQuestionAnswering.from_pretrained('bert-base-cased')
# Warning: QA head weights initialized randomly – fine-tuning needed
```

7.2 Training Arguments

```
from transformers import TrainingArguments

args = TrainingArguments(
    output_dir          = 'bert-finetuned-squad',
    evaluation_strategy = 'no',                # eval only at end
    save_strategy       = 'epoch',            # save checkpoint each epoch
    learning_rate       = 2e-5,
    num_train_epochs    = 3,
    weight_decay        = 0.01,
    fp16               = True,                # mixed precision (faster on GPU)
    push_to_hub         = True,
)
```

7.3 Launching Training

```
from transformers import Trainer

trainer = Trainer(
    model      = model,
    args       = args,
    train_dataset = train_dataset,
    eval_dataset  = validation_dataset,
    tokenizer    = tokenizer,
)

trainer.train() # ~1 hour on Titan RTX GPU
```

7.4 Evaluation After Training

```
predictions, _, _ = trainer.predict(validation_dataset)
start_logits, end_logits = predictions
compute_metrics(start_logits, end_logits,
                validation_dataset, raw_datasets['validation'])
```

Output:

```
{'exact_match': 81.18, 'f1': 88.67}
```

```
# BERT paper baseline: EM=80.8, F1=88.5 — we matched it!
```

Hyperparameter Summary

- Learning rate: $2e-5$ (standard for BERT fine-tuning)
- Epochs: 3 (sufficient for SQuAD convergence)
- Weight decay: 0.01 (L2 regularization on non-bias params)
- fp16: True (halves memory, speeds up ~30% on modern GPUs)
- Batch size: 8 (default; increase if GPU VRAM allows)

9. Using the Fine-Tuned Model

Once uploaded to the HuggingFace Hub, the model can be used directly with the pipeline API for zero-setup inference.

```
from transformers import pipeline

model_checkpoint = 'huggingface-course/bert-finetuned-squad'

qa = pipeline('question-answering', model=model_checkpoint)

context = '''
    Transformers is backed by JAX, PyTorch and TensorFlow.
    Training with one and inference with the other is seamless.
'''

question = 'Which deep learning libraries back Transformers?'

qa(question=question, context=context)
```

Output:

```
{'score': 0.9979,
 'start': 78,
 'end': 105,
 'answer': 'Jax, PyTorch and TensorFlow'}
```


10. Summary Cheat Sheet

Topic	Key Points
Task	Extractive QA: predict (start_idx, end_idx) token span as answer
Dataset	SQuAD v1.1: 87k train / 10k val; answers are context substrings
Model	bert-base-cased + linear QA head; fast tokenizer required
Input Format	[CLS] Question [SEP] Context [SEP]; max_length=384, stride=128
Train Labels	(start_token, end_token) or (0,0) if answer outside chunk
Post-process	Top-20 start/end logits; score=S+E; filter len; pick best per example
Metric	Exact Match (EM) and token-level F1; SQuAD metric from evaluate lib
Results	BERT base: EM~81.2, F1~88.6 (paper baseline: 80.8 / 88.5)
Trainer	Simple but rigid; can't run compute_metrics mid-training for QA
Accelerate	Full control; eval every epoch; multi-GPU / TPU ready; preferred

This note covers the complete pipeline for fine-tuning BERT on SQuAD using HuggingFace Transformers. The same approach extends to other encoder models (RoBERTa, ALBERT, DistilBERT) and other extractive QA datasets.